



PREMIER UNIVERSITY

Department of Computer Science & Engineering

Lab Report

Report No. : 08

Course Code : CSE 454

Course Title : Compiler Construction Laboratory

Report Title : Write a program to compute the FIRST set of a given context-free grammar.

Date of Submission : 10/21/2025

Submitted To:

Ms. Tanni Dhoom,
Assistant Professor,
Department of Computer Science and Engineering.

Submitted By:

Name	: MD Habibul Basher Saikat
ID	: 2104010202254
Program	: B.Sc. in CSE
Batch	: 40 th
Section	: C

<u>Remarks:</u>

Experiment No: 08

Experiment Name: Write a program to compute the FIRST set of a given context-free grammar.

Objectives:

- Learn what the FIRST set of a grammar symbol means.
- Write a program that computes FIRST sets for all non-terminals.
- Handle epsilon (ϵ) and productions correctly when computing FIRST.
- Understand why grammar preprocessing (remove left recursion / left factoring) is important.

Algorithm:

Prerequisites:

1. Before computing FIRST sets, make sure the grammar is in a form the algorithm can handle:
 - a. **Remove left recursion:** Left recursion can make some parsing algorithms and some set computations confusing. Convert productions like $A \rightarrow A\alpha \mid \beta$ into a non-left-recursive form (introduce a new non-terminal).
 - b. **Apply left factoring:** If two productions share a common prefix (like $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$), factor them so the grammar exposes choices cleanly. This helps deterministic computations and later LL(1) table construction.
2. Decide tokenization mode:
 - a. **Character mode:** treat each character as a token (useful for small toy grammars).
 - b. **Token mode:** treat space-separated tokens or multi-character non-terminals as tokens (preferred for real grammars).

Step-by-step FIRST algorithm:

1. **Input:** A set of productions for a grammar. Identify the set of non-terminals (N) and terminals (T). Use ϵ to denote the empty string (epsilon).
2. **Initialize:**
 - a. For each terminal t in T, set $\text{FIRST}(t) = \{ t \}$.
 - b. For each non-terminal A in N, initialize $\text{FIRST}(A) = \{ \}$ (empty set).
3. **Repeat until no change:**
 - a. For every production $A \rightarrow X_1 X_2 \dots X_k$:
 - i. If the production is empty ($A \rightarrow \epsilon$) or its right side is empty, add ϵ to $\text{FIRST}(A)$.
 - ii. Otherwise, process symbols $X_1, X_2, \dots$ from left to right:
 1. Add all terminals in $\text{FIRST}(X_1)$ except ϵ to $\text{FIRST}(A)$.
 2. If ϵ is in $\text{FIRST}(X_1)$, then continue to X_2 and add all terminals in $\text{FIRST}(X_2)$ (except ϵ) to $\text{FIRST}(A)$.
 3. Continue this way until you find an X_i that does **not** contain ϵ in its FIRST set.
 4. If every $X_1 \dots X_k$ can produce ϵ (i.e., $\epsilon \in \text{FIRST}(X_i)$ for all i), then add ϵ to $\text{FIRST}(A)$.
 - b. Repeat the entire pass until no FIRST set changes during a complete pass.
4. **Output:** FIRST sets for every non-terminal and (optionally) for terminal symbols.

Code Snippet:

```
ccl_8_2254_main.py X Test_case.txt
Compiler_construction > ccl_8_2254_main.py > detect_left_recursion
1  from collections import defaultdict
2
3  # ----- Left recursion functions -----
4  def detect_left_recursion(rules_lines):
5      print("\n--- Left Recursion Detection Result ---")
6      left_recursive_count = 0
7      parsed_rules = []
8
9  > for rule in rules_lines: ...
27
28      print(f"\nTotal rules with left recursion: {left_recursive_count}")
29      return parsed_rules, left_recursive_count
30
32  def remove_left_recursion(parsed_rules):
33      """
34      Removes left recursion from grammar using standard algorithm
35       $A \rightarrow A\alpha \mid \beta$  becomes  $A \rightarrow \beta A'$  and  $A' \rightarrow \alpha A' \mid \epsilon$ 
36      """
37      print("\n--- Left Recursion Removal Result ---")
38      grammar = {}
39      new_rules = []
40
41      # First pass: identify recursive rules
42      has_recursion = False
43  > for nt, prods in parsed_rules: ...
88
89      if not has_recursion:
90          print("\nNo left recursion found; grammar unchanged.")
91          return parsed_rules
92
93      print("\n--- Grammar after left recursion removal ---")
94  > for nt, prods in new_rules: ...
96
97      return new_rules
--
```

Figure 1: Left Recursion Code Snippet

```
100 # ----- Left factoring functions -----
101 EPSILON = '\epsilon'
102
103 > class _TrieNode: ...
107
108 > def _choose_char_mode(productions): ...
113
114 > def _tokenize_char_mode_with_nts(prod, nonterminals): ...
137
138 > def _tokenize(prod, char_mode, nonterminals=None): ...
149
150 > def _build_trie_with_mode(productions, char_mode, nonterminals=None): ...
162
163 > def _collect_maximal_prefixes(root): ...
```

```

176 > def detection_left_factoring(parsed_rules, verbose=True):
177     factoring_map = {}
178     total_groups = 0
179
180 > for nt, prods in parsed_rules: ...
205 > if verbose:
206     print(f"\nTotal left factoring groups in grammar: {total_groups}")
207     return factoring_map, total_groups
208
209 > def _unique_new_nt(base, existing_set): ...
217
218 > def removal_left_factoring(parsed_rules):
219     print("\n--- Left Factoring Removal Result ---")
220     grammar = {nt: list(prods) for nt, prods in parsed_rules}
221     existing_nts = set(grammar.keys())
222
223     iteration = 0
224 > while True: ...
286
287     print("\n--- Grammar after left factoring removal ---")
288     final_rules = []
289 > for nt, prods in grammar.items():
290     print(f"{nt} → {' | '.join(prods)}")
291     final_rules.append((nt, prods))
292     return final_rules
---
```

Figure 2: Left Factoring Code Snippet

```

295 # ----- FIRST sets -----
296 def build_tokenized_grammar(parsed_rules):
297     nonterminals = [nt for nt, _ in parsed_rules]
298     char_mode_map = {}
299     for nt, prods in parsed_rules:
300         char_mode_map[nt] = _choose_char_mode(prods)
301
302     grammar_tokens = {}
303     for nt, prods in parsed_rules:
304         cm = char_mode_map.get(nt, True)
305         tokenized_prods = []
306         current_nts = set([x for x, _ in parsed_rules])
307         for p in prods:
308             if cm:
309                 tokens = _tokenize(p, True, current_nts)
310             else:
311                 tokens = _tokenize(p, False, None)
312             tokenized_prods.append(tokens)
313         grammar_tokens[nt] = tokenized_prods
314
315     terminals = set()
316     for nt, prods in grammar_tokens.items():
317         for prod in prods:
318             for tok in prod:
319                 if tok == EPSILON:
320                     continue
321                 if tok not in grammar_tokens.keys():
322                     terminals.add(tok)
323     return grammar_tokens, char_mode_map, set(grammar_tokens.keys()), terminals
---
```

```

325 def compute_first_sets(grammar_tokens):
326     FIRST = defaultdict(set)
327
328     nonterminals = set(grammar_tokens.keys())
329     symbols = set()
330     for nt, prods in grammar_tokens.items():
331         for prod in prods:
332             for tok in prod:
333                 symbols.add(tok)
334
335     terminals = set([s for s in symbols if s not in nonterminals and s != EPSILON])
336     for t in terminals:
337         FIRST[t].add(t)
338
339     for nt in nonterminals:
340         FIRST[nt] = set()
341
342     changed = True
343 > while changed: ...
368     return FIRST
369
370 def pretty_print_first_sets(FIRST):
371     print("\n--- FIRST sets ---")
372     for sym in sorted(FIRST.keys()):
373         if sym.isprintable():
374             items = ', '.join(sorted(FIRST[sym]))
375             print(f"FIRST({sym}) = {{ {items} }}")
376
370 > def pretty_print_first_sets(FIRST):
371     print("\n--- FIRST sets ---")
372 > for sym in sorted(FIRST.keys()):
373 >     if sym.isprintable():
374         items = ', '.join(sorted(FIRST[sym]))
375         print(f"FIRST({sym}) = {{ {items} }}")
376

```

Figure 3: Follow Set Code Snippet

```

504 # -----
505 # MAIN PROGRAM
506 # -----
507 if __name__ == "__main__":
508     n = int(input("Enter number of rules: "))
509     rules = []
510     for _ in range(n):
511         rules.append(input().strip())
512
513     print("\n" + "=" * 70)
514     print("STEP 1: LEFT RECURSION DETECTION")
515     print("=" * 70)
516
517     # 1) Left recursion detection
518     parsed_rules, left_recursive_count = detect_left_recursion(rules)
519
520     print("\n" + "=" * 70)
521     print("STEP 2: LEFT RECURSION REMOVAL")
522     print("=" * 70)
523

```

```

524     # 2) Left recursion removal
525     rules_after_lr = remove_left_recursion(parsed_rules)
526
527     print("\n" + "=" * 70)
528     print("STEP 3: LEFT FACTORING DETECTION")
529     print("=" * 70)
530
531     # 3) Left factoring detection
532     factoring_map, total_fact_groups = detection_left_factoring(rules_after_lr, verbose=True)
533
534     print("\n" + "=" * 70)
535     print("STEP 4: LEFT FACTORING REMOVAL")
536     print("=" * 70)
537
538     # 4) Left factoring removal
539     if total_fact_groups > 0:
540         final_grammar = removal_left_factoring(rules_after_lr)
541     else:
542         print("\n--- Left Factoring Removal Result ---")
543         print("\nNo left factoring groups found; grammar unchanged after factoring pass.")
544         final_grammar = rules_after_lr
545
546     print("\n" + "=" * 70)
547     print("STEP 5: BUILD TOKENIZED GRAMMAR & COMPUTE FIRST SETS")
548     print("=" * 70)
549
550     # 5) Build tokenized grammar
551     grammar_tokens, char_mode_map, nonterminals, terminals = build_tokenized_grammar(final_grammar)
552
553     # Choose start symbol
554     if final_grammar:
555         start_symbol = final_grammar[0][0]
556     else:
557         start_symbol = None
558
559     print(f"\nStart Symbol: {start_symbol}")
560     print(f"Non-terminals: {sorted(nonterminals)}")
561     print(f"Terminals: {sorted(terminals)}")
562
563     # 6) Compute FIRST sets
564     FIRST = compute_first_sets(grammar_tokens)
565     pretty_print_first_sets(FIRST)
566
567     print("\n" + "=" * 70)
568     print("STEP 6: COMPUTE FOLLOW SETS")
569     print("=" * 70)
570

```

Output Snippet:

```

PS C:\Users\Habib\Desktop> & "C:\Program Files\Python314\python.exe" c:/Users/Habib/Desktop/Compiler_construction/ccl_8_2254_main.py
Enter number of rules: 3
E->E+T|T
T->T*F|F
F->(E)|id

```

```

=====
STEP 1: LEFT RECURSION DETECTION
=====

```

```

--- Left Recursion Detection Result ---
E has 1 left recursive production(s):
E → E+T
T has 1 left recursive production(s):
T → T*F
F has no left recursion.

```

```

Total rules with left recursion: 2

```

=====

STEP 2: LEFT RECURSION REMOVAL

=====

--- Left Recursion Removal Result ---

Removing left recursion from E:
Left recursive productions: E+T
Non-recursive productions: T
Introduced E':
New E $\rightarrow$ TE'
E' $\rightarrow$ +TE', ϵ

Removing left recursion from T:
Left recursive productions: T*F
Non-recursive productions: F
Introduced T':
New T $\rightarrow$ FT'
T' $\rightarrow$ *FT', ϵ

--- Grammar after left recursion removal ---

E $\rightarrow$ TE'
E' $\rightarrow$ +TE' | ϵ
T $\rightarrow$ FT'
T' $\rightarrow$ *FT' | ϵ
F $\rightarrow$ (E) | id

=====

STEP 3: LEFT FACTORING DETECTION

=====

E has no left factoring.
E' has no left factoring.
T has no left factoring.
T' has no left factoring.
F has no left factoring.

Total left factoring groups in grammar: 0

=====

STEP 4: LEFT FACTORING REMOVAL

=====

--- Left Factoring Removal Result ---

No left factoring groups found; grammar unchanged after factoring pass.

=====

STEP 5: BUILD TOKENIZED GRAMMAR & COMPUTE FIRST SETS

=====

Start Symbol: E
Non-terminals: ['E', "E'", 'F', 'T', "T'"]
Terminals: ['(', ')', '*', '+', 'd', 'i']

--- FIRST sets ---

FIRST(() = { (}
FIRST() = {) }
FIRST(*) = { * }
FIRST(+) = { + }
FIRST(E) = { (, i }
FIRST(E') = { +, ϵ }
FIRST(F) = { (, i }
FIRST(T) = { (, i }
FIRST(T') = { *, ϵ }
FIRST(d) = { d }
FIRST(i) = { i }

Discussion: The FIRST set helps us understand which terminal symbols can appear at the beginning of any string derived from a given non-terminal. In simple words, it tells us what symbols can come first when we start expanding a rule. Computing the FIRST set is very important because it is the first step toward building the LL(1) parsing table. To find the FIRST set, we look at each production and check the first symbol on the right side. If it is a terminal, that terminal directly goes into the FIRST set. If it is a non-terminal, we include all terminals from its FIRST set. If any symbol can produce ϵ (epsilon), we continue to the next symbol. Finally, if all symbols in a production can produce ϵ , then ϵ is also added to the FIRST set. This process must repeat until no more new elements can be added to any FIRST set. The result helps the parser predict which production to use when it sees a specific token. Without correct FIRST sets, it is impossible to build a correct predictive parser. That is why before constructing the parsing table, we must remove left recursion, do left factoring if needed, and correctly compute the FIRST sets for all non-terminals.